

Name: D. Sri Anjaneya
Register Number: 192425219

CO4 – AT2
MLA 0402 – Fundamentals of Machine Learning

Annexure A

Industry Problem Solving Task

1. Fraud Detection Using Bayesian Networks (20 Marks)

Industry Scenario

A digital banking organization aims to detect fraudulent credit card transactions. The available attributes include Transaction Amount, Transaction Location, Customer Login Device, and Previous Fraud History. The organization wants to predict whether a transaction is Fraudulent using a Bayesian Network.

Task

Develop a Python program to:

1. Construct a Bayesian Network representing the relationships among the variables.
2. Define the Conditional Probability Tables (CPTs).
3. Perform probabilistic inference to predict whether a transaction is fraudulent for a given customer transaction.
4. Interpret the prediction and explain how Bayesian Networks improve fraud detection in financial systems.

2. Predictive Machine Maintenance Using Hidden Markov Models (20 Marks)

Industry Scenario

A manufacturing company continuously monitors industrial machines using sensors that record Temperature, Vibration, and Motor Noise. The actual machine condition (Healthy, Warning, or Failure) is hidden. The company wants to predict the hidden machine state from sensor observations using a Hidden Markov Model (HMM).

Task

Develop a Python program to:

1. Create a Hidden Markov Model with suitable hidden states and observation sequences.
2. Define transition and emission probabilities.
3. Predict the hidden operational state of the machine for a sequence of sensor observations.
4. Explain how the predicted hidden states support predictive maintenance and reduce equipment downtime.

3. Named Entity Recognition for Customer Support Using Conditional Random Fields

Industry Scenario

A customer support company receives thousands of emails every day. The company wants to automatically identify entities such as Customer Name, Product Name, Order ID, and Issue Type from customer emails using a Conditional Random Field (CRF) model.

Task

Develop a Python program to:

1. Prepare sequential training data with appropriate word-level features.
2. Train a Conditional Random Field (CRF) model.
3. Predict entity labels for a new customer support email.
4. Evaluate the model performance and explain how CRFs improve information extraction in customer relationship management (CRM) systems.

1. Fraud Detection Using Bayesian Networks

1.1 Introduction.

A digital banking organization can use a **Bayesian Network (BN)** to detect fraudulent credit card transactions. A Bayesian Network is a probabilistic graphical model that represents relationships between different variables and calculates the probability of an event. In this system, Transaction Amount, Transaction Location, Customer Login Device, and Previous Fraud History are used to predict whether a transaction is fraudulent.

1.2 Network Structure

The Bayesian Network can be represented as:

Transaction Amount → Fraud

Transaction Location → Fraud

Login Device → Fraud

Previous Fraud History → Fraud

The system calculates:

$$P(F, r, a, ud \mid AmountLocationDeviceHistory)$$

The Conditional Probability Tables (CPTs) define the probability of fraud for different combinations of these factors.

1.3 Python Program

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

```
model = DiscreteBayesianNetwork([
    ('Amount', 'Fraud'),
    ('Location', 'Fraud'),
    ('Device', 'Fraud'),
    ('History', 'Fraud')
])
```

```
cpd_amount = TabularCPD(
    variable='Amount',
```

```
variable_card=2,  
values=[[0.70], [0.30]],  
state_names={'Amount': ['Low', 'High']}  
)
```

```
cpd_location = TabularCPD(  
    variable='Location',  
    variable_card=2,  
    values=[[0.80], [0.20]],  
    state_names={'Location': ['Normal', 'Unusual']}  
)
```

```
cpd_device = TabularCPD(  
    variable='Device',  
    variable_card=2,  
    values=[[0.75], [0.25]],  
    state_names={'Device': ['Known', 'Unknown']}  
)
```

```
cpd_history = TabularCPD(  
    variable='History',  
    variable_card=2,  
    values=[[0.85], [0.15]],  
    state_names={'History': ['No', 'Yes']}  
)
```

```
cpd_fraud = TabularCPD(  
    variable='Fraud',  
    variable_card=2,  
    values=[  
        [  
            0.99, 0.90, 0.95, 0.70,  
            0.95, 0.60, 0.75, 0.40,
```

```

        0.90, 0.50, 0.70, 0.30,
        0.80, 0.40, 0.60, 0.20
    ],
    [
        0.01, 0.10, 0.05, 0.30,
        0.05, 0.40, 0.25, 0.60,
        0.10, 0.50, 0.30, 0.70,
        0.20, 0.60, 0.40, 0.80
    ]
],
evidence=['Amount', 'Location', 'Device', 'History'],
state_names={
    'Fraud': ['No', 'Yes'],
    'Amount': ['Low', 'High'],
    'Location': ['Normal', 'Unusual'],
    'Device': ['Known', 'Unknown'],
    'History': ['No', 'Yes']
}
)

model.add_cpds(
    cpd_amount,
    cpd_location,
    cpd_device,
    cpd_history,
    cpd_fraud
)

print("Bayesian Network Valid:", model.check_model())

inference = VariableElimination(model)

result = inference.query(

```

```

variables=['Fraud'],
evidence={
    'Amount': 'High',
    'Location': 'Unusual',
    'Device': 'Unknown',
    'History': 'Yes'
}
)

print("\nFraud Detection Result:")
print(result)

```

1.4 Expected Output

Bayesian Network Valid: True

Fraud Detection Result:

```

+-----+-----+
| Fraud  | Probability |
+-----+-----+
| No     | ...        |
| Yes    | ...        |
+-----+-----+

```

The exact probabilities are calculated by the Bayesian Network based on the defined CPTs.

1.5 Interpretation

The given transaction has a **high amount, unusual location, unknown device, and previous fraud history**. These are strong indicators of suspicious activity. The Bayesian Network combines all these factors and calculates the probability of fraud. If the fraud probability is high, the transaction can be marked as suspicious and additional authentication can be requested.

1.6 Advantages in Financial Systems

- Combines multiple transaction risk factors.
- Handles uncertainty in financial data.
- Produces probability-based fraud predictions.
- Supports real-time transaction monitoring.

- Helps reduce unauthorized transactions.
- Can support additional verification for high-risk transactions.

1.7 Conclusion

Bayesian Networks provide an effective method for fraud detection because they can represent dependencies between different transaction attributes. By combining transaction amount, location, device information, and previous fraud history, the system can estimate the probability of fraud and help banks take preventive action.

2. Predictive Machine Maintenance Using Hidden Markov Models

2.1 Introduction

Manufacturing companies continuously monitor machines using sensors such as **Temperature, Vibration, and Motor Noise**. The actual condition of a machine cannot always be directly observed. It may be in a **Healthy, Warning, or Failure** state. A **Hidden Markov Model (HMM)** can estimate these hidden states from observable sensor readings.

The HMM consists of:

- **Hidden States:** Healthy, Warning, Failure
- **Observations:** Normal, Moderate, High, Critical
- **Transition Probabilities:** Probability of moving from one machine state to another.
- **Emission Probabilities:** Probability of observing a particular sensor condition from a machine state.

2.2 State Representation

Healthy → Healthy → Warning → Failure

↓ ↓ ↓ ↓

Normal Moderate High Critical

The Viterbi algorithm is used to find the most likely sequence of hidden machine states.

2.3 Python Program

```
import numpy as np
from hmmlearn import hmm
```

```
states = [
    "Healthy",
```

```
    "Warning",  
    "Failure"  
]
```

```
observations = [  
    "Normal",  
    "Moderate",  
    "High",  
    "Critical"  
]
```

```
observation_sequence = np.array([  
    [0],  
    [0],  
    [1],  
    [1],  
    [2],  
    [2],  
    [3]  
])
```

```
model = hmm.CategoricalHMM(  
    n_components=3,  
    random_state=42  
)
```

```
model.startprob_ = np.array([  
    0.70,  
    0.25,  
    0.05  
])
```

```
model.transmat_ = np.array([
```

```

    [0.80, 0.18, 0.02],
    [0.10, 0.70, 0.20],
    [0.00, 0.10, 0.90]
])

model.emissionprob_ = np.array([
    [0.75, 0.20, 0.04, 0.01],
    [0.15, 0.45, 0.30, 0.10],
    [0.02, 0.08, 0.30, 0.60]
])

log_probability, hidden_states = model.decode(
    observation_sequence,
    algorithm="viterbi"
)

print("Machine Condition Prediction")
print("-----")

for i, state in enumerate(hidden_states):
    print(
        f'Observation: {observations[observation_sequence[i][0]]}'
        f' --> Predicted State: {states[state]}'
    )

print("\nMost likely hidden state sequence:")

for state in hidden_states:
    print(states[state], end=" → ")

print("\n\nLog Probability:", log_probability)

```

2.4 Expected Output

Machine Condition Prediction:

Observation: Normal --> Predicted State: Healthy

Observation: Normal --> Predicted State: Healthy

Observation: Moderate --> Predicted State: Healthy

Observation: Moderate --> Predicted State: Warning

Observation: High --> Predicted State: Warning

Observation: High --> Predicted State: Warning

Observation: Critical --> Predicted State: Failure

Most likely hidden state sequence:

Healthy → Healthy → Healthy → Warning → Warning → Warning → Failure

2.5 Interpretation

The machine starts in a **Healthy** state when the sensor observations are normal. As the sensor readings increase from moderate to high and critical levels, the predicted state changes from **Healthy to Warning and finally Failure**.

The Warning state is particularly important because maintenance can be performed before complete machine failure occurs.

2.6 Advantages in Predictive Maintenance

- Continuously monitors machine conditions.
- Detects gradual changes in machine health.
- Predicts hidden operational states.
- Provides early warning of possible failures.
- Reduces unexpected equipment downtime.
- Helps schedule maintenance in advance.
- Reduces repair and replacement costs.
- Improves machine reliability and production efficiency.

2.7 Conclusion

HMMs are useful for predictive maintenance because machine conditions are often hidden while sensor measurements are observable. By analyzing a sequence of observations, the HMM identifies the most likely machine condition and allows maintenance teams to take action before serious equipment failure occurs.

3. Named Entity Recognition for Customer Support Using Conditional Random Fields

3.1 Introduction

Customer support companies receive a large number of emails containing useful information such as customer names, product names, order IDs, and issue descriptions. Extracting this information manually is time-consuming. **Named Entity Recognition (NER)** using a **Conditional Random Field (CRF)** can automatically identify and classify these entities.

For example:

David ordered Samsung Galaxy S24 with order ORD55555

because the screen is broken.

The CRF can identify:

Text	Entity
David	CUSTOMER
Samsung Galaxy S24	PRODUCT
ORD55555	ORDER_ID
screen broken	ISSUE

3.2 CRF Features

The model uses word-level and contextual features such as:

- Lowercase form of the word
- Word length
- Prefix and suffix
- Whether the word is uppercase
- Whether the word is a title
- Previous word
- Next word
- Beginning and end of sentence

CRFs also consider relationships between neighboring labels, making them suitable for sequential NER tasks.

3.3 Python Program

```
import sklearn_crfsuite
from sklearn_crfsuite import metrics
```

```
from sklearn.model_selection import train_test_split
```

```
def word_features(sentence, i):
```

```
    word = sentence[i]
```

```
    features = {
```

```
        'word.lower()': word.lower(),
```

```
        'word.isupper()': word.isupper(),
```

```
        'word.istitle()': word.istitle(),
```

```
        'word.isdigit()': word.isdigit(),
```

```
        'word.length': len(word),
```

```
        'prefix-2': word[:2],
```

```
        'suffix-2': word[-2:],
```

```
        'prefix-3': word[:3],
```

```
        'suffix-3': word[-3:]
```

```
    }
```

```
    if i > 0:
```

```
        previous_word = sentence[i - 1]
```

```
        features.update({
```

```
            '-1:word.lower()': previous_word.lower(),
```

```
            '-1:word.istitle()': previous_word.istitle()
```

```
        })
```

```
    else:
```

```
        features['BOS'] = True
```

```
    if i < len(sentence) - 1:
```

```
        next_word = sentence[i + 1]
```

```
        features.update({
```

```
            '+1:word.lower()': next_word.lower(),
```

```
            '+1:word.istitle()': next_word.istitle()
```

```
    })
else:
    features['EOS'] = True
```

```
return features
```

```
def sentence_features(sentence):
    return [
        word_features(sentence, i)
        for i in range(len(sentence))
    ]
```

```
sentences = [
    [
        "John", "ordered", "Samsung", "Galaxy", "S24",
        "with", "order", "ORD12345", "because",
        "screen", "broken"
    ],
    [
        "Priya", "bought", "Apple", "iPhone", "15",
        "order", "ORD67890", "battery", "problem"
    ],
    [
        "Rahul", "purchased", "Dell", "Laptop",
        "order", "ORD45678", "keyboard", "issue"
    ],
    [
        "Anita", "ordered", "Sony", "Headphones",
        "order", "ORD98765", "sound", "problem"
    ]
]
```

```

labels = [
    [
        "B-CUSTOMER", "O", "B-PRODUCT", "I-PRODUCT",
        "I-PRODUCT", "O", "O", "B-ORDER_ID",
        "O", "B-ISSUE", "I-ISSUE"
    ],
    [
        "B-CUSTOMER", "O", "B-PRODUCT", "I-PRODUCT",
        "I-PRODUCT", "O", "B-ORDER_ID",
        "B-ISSUE", "I-ISSUE"
    ],
    [
        "B-CUSTOMER", "O", "B-PRODUCT", "I-PRODUCT",
        "O", "B-ORDER_ID", "B-ISSUE", "I-ISSUE"
    ],
    [
        "B-CUSTOMER", "O", "B-PRODUCT", "I-PRODUCT",
        "O", "B-ORDER_ID", "B-ISSUE", "I-ISSUE"
    ]
]

```

```

X = [
    sentence_features(sentence)
    for sentence in sentences
]

```

```

y = labels

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,

```

```

    random_state=42
)

crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    c1=0.1,
    c2=0.1,
    max_iterations=100,
    all_possible_transitions=True
)

crf.fit(X_train, y_train)

y_pred = crf.predict(X_test)

print("CRF Model Evaluation")
print("-----")

print(
    metrics.flat_classification_report(
        y_test,
        y_pred
    )
)

new_email = [
    "David", "ordered", "Samsung", "Galaxy", "S24",
    "with", "order", "ORD55555", "and", "screen", "broken"
]

X_new = [
    sentence_features(new_email)
]

```

```

prediction = crf.predict(X_new)[0]

print("\nNamed Entity Recognition Result")
print("-----")

for word, label in zip(new_email, prediction):
    print(f'{word:15} -> {label}')

```

3.4 Expected Output

CRF Model Evaluation

precision recall f1-score

Named Entity Recognition Result

```

David      -> B-CUSTOMER
ordered    -> O
Samsung    -> B-PRODUCT
Galaxy     -> I-PRODUCT
S24        -> I-PRODUCT
with       -> O
order      -> O
ORD55555   -> B-ORDER_ID
and        -> O
screen     -> B-ISSUE
broken     -> I-ISSUE

```

The exact evaluation scores depend on the training and testing split.

3.5 Evaluation Metrics

The CRF model can be evaluated using **Precision, Recall, and F1-score**.

Precision

$$Precision = \frac{TP}{TP + FP}$$

It measures how many predicted entities are actually correct.

Recall

$$Recall = \frac{TP}{TP + FN}$$

It measures how many actual entities are successfully identified.

F1-Score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

It provides a combined measure of precision and recall.

3.6 Advantages in CRM Systems

CRFs can improve customer relationship management by:

- Automatically extracting important customer information.
- Identifying product and order details.
- Detecting customer issues from emails.
- Reducing manual data entry.
- Improving customer-support ticket classification.
- Automatically routing issues to appropriate departments.
- Saving time for customer-support employees.
- Converting unstructured emails into structured CRM data.

3.7 Conclusion

CRF-based NER provides an effective solution for extracting structured information from customer-support emails. By considering both individual word features and relationships between neighboring labels, CRFs can identify customer names, products, order IDs, and issue types. This improves automation, response time, and efficiency in CRM systems.